

Comparison Report

Unoptimized (Float32) vs. Quantized (Int8) Model on MCU

1. Introduction

We are analyzing the FOMO MobileNetV2 0.35 model, which is designed for object detection and classification. The model is optimized for edge AI applications, where computational resources are limited.

The primary goal of this model is to distinguish between two different individuals based on their visual features using an object detection approach. Instead of detecting the entire scene, **FOMO (Faster Objects, More Objects)** allows real-time object detection while maintaining a low computational footprint.

Since the model is deployed on edge devices like ESP32 and Raspberry Pi, quantization plays a key role in balancing accuracy and efficiency. This brings us to the core of our discussion: **Float32 vs. Int8 quantization**.

2. Model Architecture: Input and Output Layers

The neural network consists of the following layers:

- **Input Layer:** The model takes in **480,000 features as input**, which are extracted from the image data.
- **Hidden Layers:** MobileNetV2 typically has **53 layers** in its full version, but since this model uses MobileNetV2 0.35, it operates with only 35% of the full parameters, making it much lighter.
- **Output Layer:** The final layer consists of **three classes**, indicating the possible classifications the model can make. These classes correspond to the two individuals and the background, ensuring effective object detection.

Transfer Learning

- The model uses transfer learning, meaning it was pre-trained on a large dataset (like ImageNet) before being fine-tuned for your specific classification task.
- Instead of training from scratch, we leverage pre-trained weights and only **fine-tune the last few layers** on your dataset.
- This helps in reducing training time while maintaining high accuracy.

3. Float32 vs. Int8 Weights: Impact on Model Performance

Both models use the **FOMO (Faster Objects, More Objects) MobileNetV2 0.35** architecture, but differ in numerical precision and optimization.

Unoptimized Model (Float32)	Quantized Model (Int8)
Uses 32-bit floating-point (Float32) representation	Uses an 8-bit integer (Int8) representation.
Higher precision but requires more memory	Slightly lower precision but more memory-efficient
Slower inferencing speed on microcontrollers	Faster inferencing speed on microcontrollers.

4. Performance Metrics

Metric	Unoptimized (Float32)	Quantized (Int8)	Difference
F1 Score (Overall)	69.4%	67.9%	↓ 1.5%
Precision (Non-Background)	0.89	0.82	↓ 7%
Recall (Non-Background)	0.57	0.58	↑ 1%
F1 Score (Non-Background)	0.69	0.68	↓ 1%
Inference Time	69 ms	49 ms	↓ 30% (Faster)
Flash Usage	82.9 KB	79.4 KB	↓ 4.2% (Lower Memory Use)

5. Conclusion

Since we are working with edge devices, the Int8 model is the better choice due to its speed and memory efficiency. The slight accuracy loss is acceptable for real-time applications like person classification.